

AI Matchmaking Recommendation System

Project Documentation

- Content based & Feedback-Driven compatibility Matching using NLP and Machine Learning

1. Project Overview

The AI Matchmaking Recommendation System is a Python-based application that recommends compatible user matches for a matchmaking / dating-style platform. It combines Natural Language Processing (NLP), a rule-based personality (MBTI) compatibility model, a location-similarity heuristic, and a supervised Machine Learning model trained on historical user feedback to produce a ranked list of the top 5 most compatible matches for any selected user.

The system is delivered in two interactive forms — a Streamlit web application (APP.py) and a Tkinter desktop application (APPtk.py) — both built on the same underlying matching engine that was first prototyped in a Google Colab notebook (MAJOR_proj.ipynb / major_proj.py).

1.1 Problem Statement

Traditional matchmaking platforms often rely on simple filters — age range, location radius, a handful of tick-box preferences — which fail to capture the nuance of who a person actually is. Two people can pass every filter and still have nothing in common, while two people who would get along well can be filtered out purely because they live in different cities. This project explores a lightweight, explainable alternative: score compatibility using several independent signals (what people say about themselves, their personality type, and where they live), combine those signals into one interpretable number, and sanity-check that scoring scheme against real interaction outcomes.

1.2 Motivation

The project was built to demonstrate practical, hands-on use of NLP and machine learning techniques in a relatable, real-world-style domain (matchmaking), while keeping the system small enough to run entirely on local CSV files without any external database, paid API, or cloud infrastructure. This makes it easy to set up, inspect, and extend for learning purposes.

1.3 Scope

The current scope covers a single synthetic dataset of 80 users and 400 feedback records, a single-machine implementation with no networked backend, and two alternative front ends. It is explicitly a prototype/demonstration system rather than a production-grade matchmaking platform — see Section 7 for a full discussion of its limitations.

1.4 Objectives

- Recommend the top 5 most compatible users for any given profile based on textual, personality, and location signals.
- Quantify compatibility as a single, human-readable percentage score so results are easy to interpret and compare.
- Learn from historical user feedback (accepted vs. rejected matches) to check and

refine the weighting of matching factors, rather than trusting hand-tuned weights blindly.

- Provide an easy-to-use interface — both a browser-based option (Streamlit) and a native desktop option (Tkinter) — for exploring recommendations interactively.
- Demonstrate an end-to-end data science pipeline: synthetic data generation → data loading → feature engineering (NLP) → rule-based scoring → similarity computation → ML-based validation → interactive UI deployment.
- Keep every stage transparent and inspectable, with no black-box components, so each score can be traced back to the exact rule or model that produced it.

1.5 Intended Users

- End users of a matchmaking platform, who would use the Streamlit or Tkinter interface to browse ranked matches.
- Developers or students studying the project, who would read the notebook (MAJOR_proj.ipynb) to see the pipeline built up incrementally.
- Evaluators or reviewers who need a single authoritative reference document describing what the system does and how it works.

1.6 Key Technologies

Component	Technology / Library	Purpose in this project
Data handling	pandas	Loading, filtering, and indexing the user and feedback CSV files
Text similarity (NLP)	scikit-learn — TfidfVectorizer, cosine_similarity	Converting free-text bios into numeric vectors and measuring how similar two users' text is
Feedback model	scikit-learn — LinearRegression	Learning how strongly text similarity and MBTI compatibility actually predict a positive outcome
Web UI	Streamlit	Rendering an interactive, shareable browser-based dashboard

Desktop UI	Tkinter / ttk	Rendering a native, dependency-light desktop GUI
Prototyping environment	Google Colab (Jupyter Notebook)	Iterative development, experimentation, and CLI-style testing of the pipeline
Randomised data generation	Python's random and datetime modules	Producing the synthetic user.csv and feedback.csv datasets used to demo the system

2. Data Sources

The system relies on two flat CSV files that sit alongside the application code. Both are generated synthetically by dedicated helper scripts so the project can be demonstrated end-to-end without needing access to a real user base. This section documents each file in detail, including how it is produced, its exact schema, and example values.

2.1 user.csv — User Profiles

user.csv contains synthetic profile data for 80 users (IDs X01 through X80), generated by user.py. Each row represents one person on the matchmaking platform, describing who they are, what they do, and how they describe themselves in free text.

2.1.1 Generation Process

user.py builds the dataset as a plain Python dictionary of parallel lists — one list per column, all the same length (80) — which is then passed directly to `pandas.DataFrame()` and written out with `data.to_csv("user.csv", index=False)`. Each list (names, ages, locations, professions, MBTI types, interests, etc.) was hand-curated to look like plausible, varied profiles: 80 distinct first names, 80 distinct Indian cities (e.g. Bangalore, Hyderabad, Chennai, Pune, Delhi, Mumbai, Gurgaon, ... down to Tirupati), and 80 distinct professions spanning engineering, business, healthcare, creative, and legal fields. Ages range from about 20 to 45. Because every list is independently authored, the mapping between fields (e.g. which profession goes with which MBTI type) is effectively arbitrary/random rather than psychologically modelled — this is a synthetic demo dataset, not real survey data.

2.1.2 Schema

Column	Data type	Description
user_id	string	Unique identifier in the form X01–X80, used as the primary key throughout the pipeline
name	string	First name of the user (e.g. Aarav, Vivaan, Advik)
age	integer	Age in years (observed range roughly 20–45 across the dataset)
location	string	City of residence — one of 80 distinct Indian cities used across the dataset
profession	string	Job title (e.g. Software Engineer, Data Analyst, Project Manager)
exp_years	integer	Years of professional experience
professional_summary	string (free text)	A short description of the user's professional background, e.g. "Developing scalable backend systems using Python and microservices"
about_me	string (free text)	A short personal bio, e.g. "Passionate about continuous learning and personal growth"
MBTI	string	Myers-Briggs personality type, one of the 16 four-letter codes (e.g. INTJ, ENFP, INTP)
intrests	string (comma-separated list)	Hobbies/interests, e.g. "Data, Fitness, Teaching" (note: column name is spelled

'intrests' in the raw file, not 'interests')

--	--	--

2.1.3 Sample Records

user_id	name	age	location	MBTI
X01	Aarav	40	Bangalore	INTP
X02	Vivaan	20	Hyderabad	INTJ
X03	Advik	20	Chennai	ENTJ
X04	Reyansh	43	Pune	ESTJ

2.1.4 How This File Is Used

professional_summary and about_me are the two fields feeding the NLP/TF-IDF text-similarity component (Section 4.1). MBTI feeds the personality-compatibility lookup (Section 4.2). location feeds the location-matching heuristic (Section 4.3). name, profession, and MBTI are also displayed back to the user in the "Selected User Profile" panel of the applications (Section 5).

2.2 feedback.csv — Interaction History

feedback.csv contains 400 historical match-feedback records generated by feedback.py. It simulates a log of past match suggestions and how each user responded to them, which is the data the Linear Regression validation model (Section 4.5) is trained on.

2.2.1 Generation Process

For each of the 80 users (X01–X80), feedback.py generates exactly 5 feedback rows ($80 \times 5 = 400$ total). For each row, a random matched_user_id is drawn uniformly from the same X01–X80 pool, with a while-loop guarding against a user being matched to themselves (matched_user == user_id is re-rolled until it differs). The action label is chosen uniformly at random between 0 (rejected) and 1 (accepted) using random.choice([0, 1]). A random date between 2025-01-01 and 2025-12-31 is generated by picking a random integer offset (in days) within that range and adding it to the start date. All rows are collected into a list of lists, converted into a pandas DataFrame with

columns [user_id, matched_user_id, action, timestamp], and written to feedback.csv.

2.2.2 Schema

Column	Data type	Description
user_id	string	The user who received the suggested match (X01–X80)
matched_user_id	string	The suggested match's user_id — guaranteed to differ from user_id
action	integer (0 or 1)	Feedback label: 1 = accepted/liked, 0 = rejected/passed
timestamp	date string (YYYY-MM-DD)	Date the feedback was recorded, randomly distributed across 2025

2.2.3 Sample Records

user_id	matched_user_id	action	timestamp
X01	X37	0	2025-04-16
X01	X02	0	2025-08-07
X01	X07	1	2025-03-02
X01	X50	0	2025-01-18

2.2.4 Important Caveat

In the current version, `feedback.csv` is randomly generated purely for demonstration purposes — the action label is not actually derived from any of the compatibility factors, so it carries no real signal about what makes a good match. In a production deployment, this table would instead be populated by genuine user interactions (e.g. `swipe-right/left`, `message-sent`, `date-accepted` events captured by the application backend), which would give the Linear Regression model in Section 4.5 real predictive power rather than a statistical exercise.

2.3 Data Volume Summary

File	Rows	Columns	Generated by
<code>user.csv</code>	80	10 (+ index)	<code>user.py</code>
<code>feedback.csv</code>	400	4	<code>feedback.py</code>

3. System Architecture

The pipeline follows four sequential stages, each of which is described in depth below and expanded further in Section 4 (Methodology).

Stage 1 — Data Ingestion

Both `APP.py` and `APPtk.py` begin by loading `user.csv` and `feedback.csv` into pandas DataFrames with `pd.read_csv()`. This happens once, at application start-up, and the resulting DataFrames are kept in memory for the lifetime of the running process — there is no database connection or repeated disk I/O per request.

Stage 2 — Feature Engineering (NLP)

Immediately after loading, the users DataFrame gains a new derived column, `combined_text`, formed by concatenating `professional_summary` and `about_me` (with missing values replaced by empty strings via `.fillna("")`). This combined text is passed once through a `TfidfVectorizer` to build a single `tfidf_matrix` covering all 80 users — this matrix is computed once and reused for every subsequent compatibility calculation, which keeps the app responsive.

Stage 3 — Compatibility Scoring

Whenever a compatibility score is needed between two specific users, the `compatibility(user1, user2)` function looks up each user's row index, retrieves their pre-computed TF-IDF vector, MBTI code, and location, and combines the three sub-scores (text, MBTI, location) using the fixed weights $w_1/w_2/w_3$ into a single 0–100 compatibility percentage.

Stage 4 — Feedback Validation & Recommendation Serving

Separately, all 400 rows of `feedback.csv` are converted into a training set ($X = [\text{text_similarity}, \text{mbti_score}]$, $y = \text{action}$) and used to fit a scikit-learn `LinearRegression` model. This model does not replace the hand-tuned `compatibility()` scoring — it runs in parallel, purely to report back how much weight text similarity and MBTI compatibility actually carry in predicting a historical accept/reject outcome. Finally, `top_matches(user_id)` loops `compatibility()` over every other user, sorts descending by score, and returns the top 5 — these are what the UI layer (Streamlit or Tkinter) renders to the end user.

3.1 Data Flow Summary

- `user.csv + feedback.csv` (disk) → pandas DataFrames (memory)
- `users.professional_summary + users.about_me` → `combined_text` → `TfidfVectorizer` → `tfidf_matrix`
- `tfidf_matrix + MBTI lookup + location comparison` → `compatibility(user1, user2)` → single score
- `feedback.csv rows + tfidf_matrix + MBTI lookup` → `LinearRegression.fit()` → learned coefficients
- `compatibility()` looped over all users → `top_matches(user_id)` → sorted top-5 list
- top-5 list + selected user's profile → rendered in Streamlit table/chart or Tkinter Treeview

3.2 File Map

The project is organised into data-generation scripts, a prototyping notebook, and two parallel production front ends, all sharing the same core matching logic.

File	Role
<code>user.py</code>	Generates the synthetic <code>user.csv</code> profile dataset (80 users) by assembling hand-authored Python lists into a DataFrame and exporting to CSV.
<code>feedback.py</code>	Generates the synthetic <code>feedback.csv</code> interaction dataset (400 records: 5 per user) using randomised match partners, actions, and dates.
<code>MAJOR_proj.ipynb / major_proj.py</code>	Original Google Colab notebook — the exploratory prototype where the full pipeline (data load → NLP → MBTI rules → compatibility function → feedback model) was first built up cell by cell, ending in an interactive

	command-line lookup.
APP.py	Production Streamlit web application — wraps the same matching engine in a browser-based dashboard with a dropdown, results table, and bar chart.
APPtk.py	Production Tkinter desktop application — the same matching engine wrapped in a native desktop GUI with a combobox, results table (Treeview), and live display of the learned regression weights.

3.3 Design Principles

- Single source of truth for scoring logic — `compatibility()`, `mbti_score()`, and `location_score()` are duplicated verbatim between APP.py and APPtk.py rather than imported from a shared module, which keeps each UI file self-contained and independently runnable, at the cost of needing any future logic change to be applied in two places.
- In-memory, stateless computation — no results are cached to disk between runs; every launch reloads the CSVs and recomputes the TF-IDF matrix and regression model fresh.
- Separation of concerns — data generation (`user.py`, `feedback.py`), exploratory analysis (the notebook), and presentation (APP.py / APPtk.py) are kept in separate files.

4. Methodology

4.1 Text Similarity (NLP)

Each user's `professional_summary` and `about_me` fields are concatenated into a single `combined_text` field, giving a compact document that represents "how this person describes themselves" in their own words. This corpus of 80 documents is transformed into TF-IDF (Term Frequency–Inverse Document Frequency) vectors using scikit-learn's `TfidfVectorizer`.

4.1.1 What TF-IDF Does

TF-IDF represents each document as a vector where every dimension corresponds to a word in the overall vocabulary. A word's weight in a document is high when that word appears often in that document (term frequency) but rarely across the whole collection of documents (inverse document frequency) — so common words that appear in almost everyone's bio ("passionate", "enjoys", "technology") get down-weighted, while

distinctive words that set a person apart get more weight. The `stop_words="english"` setting additionally strips out grammatical filler words ("the", "and", "of", etc.) before this weighting is even applied.

4.1.2 Cosine Similarity

Once every user is represented as a TF-IDF vector, comparing two users' text is a matter of measuring the angle between their two vectors using cosine similarity. A value of 1.0 (100%) means the two texts point in an identical direction (near-identical wording/topics); a value of 0.0 means the two texts share no meaningful vocabulary overlap. The system multiplies this 0–1 similarity by 100 to express it as a percentage, consistent with the other two sub-scores.

```
users["combined_text"] = users["professional_summary"].fillna("") + " " +  
users["about_me"].fillna("")  
vectorizer =  
TfidfVectorizer(stop_words="english")  
tfidf_matrix =  
vectorizer.fit_transform(users["combined_text"])  
text_score =  
cosine_similarity(tfidf_matrix[idx1], tfidf_matrix[idx2])[0][0] * 100
```

This TF-IDF matrix is computed once, when the application starts, and reused for every pairwise comparison thereafter — recomputing it for every single `compatibility()` call would be wasteful, since the vocabulary and weighting do not change between calls.

4.2 MBTI Personality Compatibility

The Myers-Briggs Type Indicator (MBTI) sorts personalities into 16 four-letter types along four axes: Introversion/Extraversion, iNtuition/Sensing, Thinking/Feeling, and Judging/Perceiving. Certain type pairings are popularly considered to be especially complementary because they balance each other's strengths and weaknesses.

4.2.1 Lookup Table

The system encodes this idea as a Python dictionary keyed by `(type_a, type_b)` tuples, mapping specific pairings to a compatibility score out of 100. Only two complementary pairings are currently modelled explicitly: INTJ↔ENFP (100 — the archetypal "strategist and inspirer" pairing) and INFJ↔ENTP (95 — the "advocate and debater" pairing). Both directions of each pair are stored, since `(A, B)` and `(B, A)` are different dictionary keys. Any pairing not explicitly listed — which, given only 4 of the possible 256 ordered type combinations are covered, is the vast majority of cases — falls back to a flat baseline score of 60 via `dict.get()`'s default value.

```
MBTI_SCORES = { ("INTJ", "ENFP"): 100, ("ENFP", "INTJ"): 100, ("INFJ",  
"ENTP"): 95, ("ENTP", "INFJ"): 95}  
def mbti_score(a, b):  
    return  
    MBTI_SCORES.get((a, b), 60)
```

4.2.2 Interpretation

Because so few pairings are explicitly scored, the MBTI factor mostly acts as a mild tie-breaker rather than a fine-grained signal: most user pairs receive the same default 60, and only the two special pairings stand out with materially higher scores.

4.3 Location Compatibility

Location is treated with the simplest possible heuristic — an exact string match on the location column. If both users list the same city, the pair scores 100 (maximum proximity benefit); if the cities differ at all, the pair scores a flat 50, regardless of whether the two cities are neighbouring towns or on opposite sides of the country.

```
def location_score(a, b):    if a == b:        return 100    return 50
```

This binary same-city / different-city heuristic is intentionally simple; Section 7 discusses replacing it with a distance-aware alternative.

4.4 Weighted Compatibility Score

The three independent sub-scores — text_score, mbti, and location — are combined into a single compatibility percentage using a fixed linear weighting scheme chosen by the project author to reflect an assumed order of importance.

Factor	Weight	Rationale
Text similarity (bio/profession)	w1 = 0.5	Treated as the strongest signal — shared interests, values, and professional background expressed in each user's own words
MBTI compatibility	w2 = 0.3	Personality complementarity, as a secondary factor
Location match	w3 = 0.2	Practical/logistical proximity, treated as the least important of the three

```
total = w1 * text_score + w2 * mbti + w3 * locationreturn round(total, 2)
```

4.4.1 Worked Example

Suppose two users have a text similarity of 40% (text_score = 40), share no special MBTI pairing (mbti = 60), and live in the same city (location = 100). The final compatibility score would be: $0.5 \times 40 + 0.3 \times 60 + 0.2 \times 100 = 20 + 18 + 20 = 58.0\%$. If the same two users instead had an MBTI pairing of INTJ/ENFP (mbti = 100) while everything else stayed the same, the score would rise to $0.5 \times 40 + 0.3 \times 100 + 0.2 \times 100 = 20 + 30 + 20 = 70.0\%$ — illustrating how a strong personality match can meaningfully lift an otherwise

average pairing.

4.5 Feedback-Based Validation Model

Hand-tuned weights (0.5 / 0.3 / 0.2) are a reasonable starting assumption, but they are still a guess. To check that guess against evidence, the system trains a separate, independent Linear Regression model on the 400 rows of feedback.csv, treating each row as one observed outcome of a suggested match.

4.5.1 Building the Training Set

For every feedback row, the system looks up the two users involved (user_id and matched_user_id), computes their text similarity (as a 0–1 cosine similarity, not yet scaled to 100) and their MBTI compatibility score (normalised to 0–1 by dividing by 100), and uses that two-element vector as the input features (X). The actual recorded action (1 = accepted, 0 = rejected) becomes the target label (y). Location is deliberately excluded from this particular model — only text similarity and MBTI compatibility are used as predictors.

```
text_sim = cosine_similarity(tfidf_matrix[idx1], tfidf_matrix[idx2])[0][0]mbti =
mbti_score(users.loc[idx1, "MBTI"], users.loc[idx2, "MBTI"]) / 100X.append([text_sim,
mbti])y.append(row["action"])model = LinearRegression()model.fit(X, y)# model.coef_[0]
-> learned weight for text similarity# model.coef_[1] -> learned weight for MBTI
compatibility
```

4.5.2 What the Coefficients Mean

After fitting, model.coef_[0] and model.coef_[1] represent how much the predicted probability of acceptance changes for a one-unit increase in text similarity and MBTI score respectively, holding the other constant. A larger positive coefficient means that factor is a stronger real-world predictor of a positive outcome, at least within this feedback dataset. Comparing these learned coefficients to the fixed w1/w2 weights used in compatibility() is a quick, informal way to gauge whether the hand-tuned assumptions (text matters more than personality) are supported by the data.

In APPTk.py, these learned coefficients are displayed live on the desktop UI as "Text Weight" and "MBTI Weight" so a user can visually compare the model's learned importance against the hand-tuned weights every time they run a search. The original notebook (major_proj.py) also prints these coefficients directly to the console for inspection.

4.5.3 Important Caveat

Because feedback.csv (Section 2.2) is currently generated with a purely random action label — one that has no real relationship to text similarity or MBTI — the coefficients learned by this model on the shipped dataset are not meaningful in a statistical sense; they mainly demonstrate how the validation mechanism would work once genuine feedback data is available.

4.6 Top-5 Recommendation Generation

For a selected user, the `top_matches(user_id)` function iterates over every other `user_id` in the dataset, calls `compatibility()` for each pair, and collects the results as a list of `[uid, score]` entries. This list is sorted in descending order of score using Python's `sort(key=..., reverse=True)`, and the first five entries are returned as the user's top matches. With 80 users this means at most 79 `compatibility()` calls per lookup — fast enough to run interactively without any additional caching or indexing.

```
def top_matches(user_id):
    scores = []
    for uid in users["user_id"]:
        if uid != user_id:
            score = compatibility(user_id, uid)
            scores.append([uid, score])
    scores.sort(key=lambda x: x[1], reverse=True)
    return scores[:5]
```

5. Application Interfaces

The matching engine described in Section 4 is exposed to end users through three distinct interfaces, each suited to a different context: a shareable web dashboard, a native desktop tool, and a bare command-line prototype. All three call the same underlying `compatibility()` and `top_matches()` logic; they differ only in how results are presented and interacted with.

5.1 Streamlit Web App (APP.py)

APP.py wraps the matching engine in Streamlit, a Python framework that turns a script into an interactive web page without requiring any HTML/CSS/JavaScript. The whole script re-runs top to bottom each time the user interacts with a widget, which is why the data loading and model training happen unconditionally at the top of the file.

5.1.1 Page Configuration

- `st.set_page_config()` sets the browser tab title to "AI Matchmaking System" and switches the page to a wide layout, giving the results table and chart more horizontal room.
- A large title, "💖 AI Matchmaking Recommendation System", and a short instructional line ("Find the top 5 compatible matches.") introduce the page.

5.1.2 User Interaction Flow

1. A `st.selectbox` dropdown lists every `user_id` from `users["user_id"]`, letting the visitor pick whose matches they want to see.
2. Clicking the "Find Matches" button (`st.button`) triggers the block that calls `top_matches(selected_user)`.
3. The returned list of `[uid, score]` pairs is converted into a small pandas DataFrame with columns "Matched User" and "Compatibility Score".
4. A `st.success()` banner confirms whose matches are being shown (e.g. "Top Matches for X01").
5. The results DataFrame is rendered as an interactive, sortable table via `st.dataframe(..., use_container_width=True)`.
6. The same results are also plotted as a `st.bar_chart`, indexed by "Matched User", giving an at-a-glance visual comparison of the five compatibility scores.

7. Finally, a "Selected User Profile" subheader introduces a second small table showing the chosen user's name, location, profession, and MBTI type, so the visitor has context for who they just searched.

5.1.3 Running the App

From a terminal in the project directory (with user.csv and feedback.csv present):

```
streamlit run APP.py
```

Streamlit starts a local web server (by default at <http://localhost:8501>) and automatically opens the page in the default browser. Any code change to APP.py can be picked up by using Streamlit's built-in "Rerun" option in the browser.

5.2 Tkinter Desktop App (APPtk.py)

APPtk.py implements the identical matching logic — the same combined_text construction, TF-IDF vectorisation, MBTI table, location heuristic, weighted compatibility() function, and LinearRegression feedback model — but presents it through Tkinter, Python's built-in native GUI toolkit, so the tool can run as a standalone desktop window with no browser or internet connection required.

5.2.1 Window Layout

- The root window is titled "AI Matchmaking System", sized 900×600 pixels, and given a light-grey (#f5f5f5) background.
- A bold, dark-blue heading ("AI Matchmaking Recommendation System") sits at the top of the window.
- A horizontal frame beneath the heading holds a "Select User ID:" label, a ttk.Combobox pre-populated with every user_id, and a green "Find Matches" button.
- Below that, a ttk.Treeview table (styled as a two-column grid: "Matched User" and "Compatibility Score (%)", each 200px wide) displays the results.
- A red, bold weight_label beneath the table shows the live Linear Regression coefficients.
- A footer label ("Machine Learning Based Matchmaking System") is pinned to the bottom of the window.

5.2.2 Interaction Flow (find_matches())

1. The user selects a User ID from the combobox and clicks "Find Matches", which triggers the find_matches() callback.
2. If no user was selected, a messagebox.showerror() dialog prompts the user to make a selection, and the function returns early.
3. Any rows already shown in the Treeview are cleared (tree.delete(row) for every existing child) so results don't stack up across repeated searches.
4. top_matches(selected_user) is called, and each (uid, score) pair is inserted as a new row into the Treeview via tree.insert().

5. The `weight_label` text is updated to show the current model's learned coefficients, formatted as "Text Weight: <value> MBTI Weight: <value>", each rounded to 3 decimal places.

5.2.3 Running the App

```
python APPtk.py
```

This launches the Tkinter main loop (`root.mainloop()`) and opens the desktop window directly — no server or browser is involved, which makes this variant convenient for offline demonstrations.

5.3 Notebook / CLI Prototype (MAJOR_proj.ipynb, major_proj.py)

The original Google Colab notebook is where the entire pipeline was first designed and tested, one cell at a time, before being packaged into the two applications above. Reading it top to bottom tells the story of how the project evolved:

1. Load `user.csv` and `feedback.csv` and preview the data with `users.head()`.
2. Build `combined_text` and fit the `TfidfVectorizer` to create `tfidf_matrix`.
3. Define the `MBTI_SCORES` lookup table, `mbti_score()`, and `location_score()` helper functions.
4. Write an early version of `compatibility()` (note: an initial draft in the notebook references a lower-case "mbti" column before being corrected to the actual "MBTI" column name — visible directly in the notebook history as a fixed bug).
5. Define `top_matches()` and test it directly against a hard-coded ID (`top_matches("X01")`).
6. Build the training set from `feedback.csv`, fit the `LinearRegression` model, and print the learned coefficients for Text Similarity and MBTI to the console.
7. End with an interactive console prompt: `input("Enter User ID: ")` reads a user-typed ID, validates it against `users["user_id"].values`, and prints the top 5 matches with their scores directly to the console (or an "Invalid User ID!" message if the input doesn't exist).

This CLI-style ending makes `major_proj.py` runnable directly from a terminal as a minimal, dependency-light way to query the system without any GUI at all:

```
python major_proj.pyEnter User ID: X01
```

5.4 Comparing the Three Interfaces

Interface	Best for	Requires
Streamlit (APP.py)	Shareable, visual, browser-based demos with charts	streamlit package + a browser

Tkinter (APPTk.py)	Offline desktop use, no browser dependency, model-weight transparency built in	Tkinter (bundled with most Python installs)
Notebook/CLI (major_proj.py)	Step-by-step learning, debugging, and quick console queries	Just Python + pandas/scikit-learn

6. Setup & Usage

6.1 Prerequisites

- Python 3.8 or later installed and available on the command line.
- pip for installing the required Python packages.
- Tkinter is required only for APPTk.py — it ships with most standard Python installations on Windows and macOS; on some Linux distributions it must be installed separately (e.g. `sudo apt install python3-tk`).

6.2 Installing Dependencies

```
pip install pandas scikit-learn streamlit
```

This installs pandas (data handling), scikit-learn (TfidfVectorizer, cosine_similarity, LinearRegression), and streamlit (for the web app). No other third-party packages are required — Tkinter and its ttk/messagebox submodules are part of the Python standard library.

6.3 Generating the Data

The two datasets must exist before any of the applications will run, since all three read user.csv and feedback.csv directly from the working directory.

1. Run `python user.py` — this creates user.csv containing 80 synthetic user profiles.
2. Run `python feedback.py` — this creates feedback.csv containing 400 synthetic feedback records (5 per user).

Both scripts print a short confirmation to the console when they finish (feedback.py additionally prints `feedback_df.info()` and the full DataFrame as a sanity check).

6.4 Project Directory Layout

For any of the three interfaces to run without modification, the CSV files and the corresponding script must sit in the same folder:

project/├─ user.py├─ feedback.py├─ user.csv (generated)├─ feedback.csv
(generated)├─ major_proj.py (or MAJOR_proj.ipynb)├─ APP.py├─ APPtk.py

6.5 Running the Application

1. Ensure user.csv and feedback.csv are present in the same directory as the app script (regenerate them per Section 6.3 if needed).
2. Launch the web app with streamlit run APP.py, or the desktop app with python APPtk.py, or the console prototype with python major_proj.py.
3. In the web or desktop app, select a User ID from the dropdown/combobox and click "Find Matches" to view the top 5 recommendations, the compatibility chart/table, and the selected user's profile. In the console prototype, type a User ID when prompted.
4. To try a different user, simply repeat step 3 — no restart is required in either GUI application.

6.6 Troubleshooting

Symptom	Likely Cause / Fix
FileNotFoundError: user.csv	Run python user.py first; ensure the app is launched from the same folder as the CSV files.
KeyError on "MBTI" or "mbti"	Column names are case-sensitive — the profile file uses "MBTI" (uppercase); ensure any custom data matches this exactly.
ModuleNotFoundError: streamlit / sklearn	Re-run the pip install command in Section 6.2 inside the active Python environment.
Tkinter window doesn't open on Linux	Install the Tk bindings for your Python version, e.g. sudo apt install python3-tk.
Combobox / dropdown is empty	Confirm user.csv was generated successfully and has data (check with a text editor or pandas).

7. Limitations & Future Work

7.1 Current Limitations

Data limitations

- feedback.csv is randomly generated rather than collected from real user behaviour, so the action label carries no genuine relationship to text similarity or MBTI — the trained regression weights are therefore not meaningful for production decision-making yet (see Section 4.5.3).
- user.csv is a small, synthetic dataset of 80 profiles built from hand-curated lists rather than real demographic or psychometric data, so the diversity and realism of matches is limited to what those lists contain.
- The dataset does not model any user preferences (e.g. desired age range, gender preference, deal-breakers), which a real matchmaking system would need.

Modelling limitations

- MBTI compatibility scores are hard-coded for only two type pairs (INTJ↔ENFP, INFJ↔ENTP); all other 250+ ordered combinations default to a flat 60, making the MBTI factor a coarse tie-breaker rather than a fine-grained signal.
- Location scoring is binary (same city = 100 / different city = 50) and ignores geographic distance entirely — a user 5 km away and one 2,000 km away score identically.
- The compatibility weights ($w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$) are manually chosen rather than learned end-to-end from outcome data, so they encode the author's assumptions rather than validated importance.
- TF-IDF with cosine similarity captures surface-level vocabulary overlap; it does not understand synonyms, sentiment, or deeper semantic meaning (e.g. "loves hiking" and "enjoys the outdoors" would not be recognised as related).
- The Linear Regression feedback model assumes a simple linear relationship between the two input features and outcome probability, and does not account for interaction effects (e.g. text similarity mattering more when MBTI compatibility is already high).

Engineering limitations

- No persistence layer — recommendations are recomputed from flat CSV files on every application run rather than served from a database, and there is no caching of computed scores between sessions.
- compatibility(), mbti_score(), and location_score() are duplicated verbatim across APP.py, APPtk.py, and major_proj.py rather than centralised in a shared module, so any bug fix or logic change must currently be applied in multiple places.
- There is no automated test suite covering the scoring functions, the data-generation scripts, or the UI logic.
- The system is single-user/single-machine — there is no concept of accounts, authentication, or concurrent access from multiple people.

7.2 Suggested Improvements

Data & feedback

- Collect genuine user interaction data (swipe/accept/reject events with timestamps) to replace the randomly generated feedback.csv, giving the Linear Regression model real predictive power.
- Expand user.csv with additional fields such as stated preferences, deal-breakers, and richer demographic attributes.

Modelling

- Replace TF-IDF with contextual sentence embeddings (e.g. Sentence-BERT or a similar transformer-based encoder) for richer, meaning-aware semantic matching that understands synonyms and paraphrasing.
- Extend MBTI scoring to a full 16×16 compatibility matrix grounded in established psychological research, rather than two hard-coded pairs and a flat default.
- Use geocoded distance (e.g. haversine distance between city coordinates, or actual travel time) instead of exact-city string matching for the location factor.
- Learn the weighting scheme ($w_1/w_2/w_3$) directly from feedback data — for example by using the coefficients of the Section 4.5 regression model (once trained on real feedback) as the production weights instead of hand-tuned constants.
- Experiment with more expressive models than Linear Regression (e.g. Logistic Regression, which is more appropriate for a binary 0/1 target, or gradient-boosted trees) for the feedback validation step.

Engineering & deployment

- Refactor the shared scoring logic (compatibility, mbti_score, location_score, TF-IDF setup) into a single importable module used by all three interfaces, eliminating code duplication.
- Migrate from flat CSV files to a proper database (e.g. PostgreSQL or MongoDB) with an API layer, enabling multi-user access and persistent storage of feedback.
- Add authentication, consent handling, and privacy safeguards before this system could ever process real personal data.
- Introduce automated tests for the scoring functions and data pipeline to guard against regressions as the codebase grows.
- Add caching (e.g. precomputing and storing the TF-IDF matrix and pairwise scores) to scale beyond a few hundred users.

8. Summary

This project demonstrates a complete, if lightweight, recommendation-system pipeline: synthetic data generation, TF-IDF based text similarity, rule-based personality and location scoring, a weighted compatibility formula, feedback-driven model validation via Linear Regression, and three parallel interfaces — a Streamlit web app, a Tkinter

desktop app, and a console/notebook prototype. Every stage of the pipeline, from raw CSV files to the final ranked recommendation list, is transparent and traceable to a specific, documented rule or model, which makes the system easy to understand, audit, and extend. It serves as a solid educational foundation that can be built upon with real data, more sophisticated NLP models, and a learned — rather than hand-tuned — weighting scheme, as outlined in Section 7.2.

9. Glossary

Term	Meaning
TF-IDF	Term Frequency–Inverse Document Frequency; a numeric representation of text that highlights words distinctive to a document relative to a collection.
Cosine similarity	A measure of how similar two vectors are, based on the angle between them; ranges from 0 (unrelated) to 1 (identical direction).
MBTI	Myers-Briggs Type Indicator; a 16-category personality typology used here as a compatibility heuristic.
Compatibility score	The single 0–100 percentage produced by combining text, MBTI, and location sub-scores using fixed weights.
Linear Regression	A statistical model that fits a straight-line (linear) relationship between input features and a numeric/binary target, used here to validate the hand-tuned weights.
Feature engineering	The process of transforming raw data (e.g. free-text bios) into numeric inputs (e.g. TF-IDF vectors) usable by a model.
Top-5 matches	The five other users with the highest compatibility() score relative to a selected user.